МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт Институт компьютерных наук
Кафедра Автоматизированные системы управления

ЛАБОРАТОРНАЯ РАБОТА № 1

Реализация экспертной системы с использованием прямой цепочки рассуждений

По дисциплине: «Системы искусственного интеллекта»

Студент ПИ-23-1 (группа) _____ (подпись, дата) Сорокин В.И. (фамилия, инициалы)

Руководитель

ассистент (ученая степень, ученое звание) _____ (подпись, дата) Первушин О.С. (фамилия, инициалы)

Липецк 2026

1. Задание кафедры

Разработать оболочку экспертной системы, реализующую прямую цепочку рассуждений, на любом языке программирования.

Правила должны формироваться в виде «ЕСЛИ Объект1=Значение1 ТО Объект2=Значение2». Допускается использование логической связки «И».

База правил должна быть отделена от программного модуля и представлять собой отдельный текстовый файл.

Должна быть реализована возможность редактирования базы правил: изменения, дополнения и удаления правил.

В программе должны быть предусмотрены средства для отображения состояния рабочей базы данных.

Задать стартовую ситуацию — начальное состояние рабочей базы данных.

Если ни одно из правил не может сработать, программа должна запрашивать у пользователя новые сведения о некотором объекте. Если таких сведений нет, программа завершает работу.

Выбрать предметную область, составить базу правил и протестировать работу программы.

Экспертная система на основе стартовой ситуации и базы правил должна сделать определённый вывод и отобразить его на экране.

Выбранная предметная область — диагностика неисправностей персонального компьютера.

2. Описание алгоритма

Система состоит из базы знаний rules.txt, рабочей памяти фактов, механизма прямого вывода и интерфейса консультации. В начале рабочая память пуста: фиксированного списка вопросов в программе нет.

Правила содержат одно или несколько условий, соединённых связкой «И», и заключение. Заключение сработавшего правила становится новым фактом рабочей памяти и может использоваться в следующем правиле. Так формируются цепочки вывода. Программа раскрывает промежуточные заключения и выбирает вопросы, необходимые для получения целевого объекта «Диагноз».

1. Загрузить базу правил из файла rules.txt.

2. Начать консультацию с пустой рабочей базой фактов.

3. Проверить условия правил по фактам рабочей базы.

4. Если условия совпали, добавить заключение правила как новый факт.

5. Повторять прямой проход, пока появляются новые факты; при появлении «Диагноз» вывести результат.

6. Если новых фактов нет, определить цепочки правил, ведущие к диагнозу, и найти неизвестные условия.

7. Запросить неизвестный факт из кратчайшей подходящей цепочки. Если длины цепочек совпадают, учитывается их порядок в rules.txt. Ответ добавить в рабочую базу и возобновить вывод; пустой ответ завершает консультацию.

8. Показать рабочую базу, трассировку правил и результат.

3. База правил

База знаний отделена от программного модуля и хранится в файле rules.txt. В текущей версии 20 правил, объединённых в несколько диагностических цепочек. В таблице приведены условия и заключения каждой ветви.

Условие	Заключение
Компьютер_включается=да И Экран_черный=да	Симптом=изображение_не_появляется
Компьютер_включается=нет И Индикаторы_горят=нет	Симптом=нет_признаков_питания
Симптом=нет_признаков_питания И Вентиляторы_вращаются=нет	Предполагаемая_причина=проблема_с_блоком_питания
Компьютер_включается=нет И Индикаторы_горят=да	Предполагаемая_причина=проблема_кнопки_или_системной_платы
Симптом=изображение_не_появляется И Звук_загрузки=да	Предполагаемая_причина=система_загрузилась_без_вывода_изображения
Симптом=изображение_не_появляется И Звук_загрузки=нет	Состояние_загрузки=система_не_загрузилась
Состояние_загрузки=система_не_загрузилась И Сигналы_ошибки=да	Предполагаемая_причина=сбой_видеокарты_или_POST
Предполагаемая_причина=система_загрузилась_без_вывода_изображения И Изображение_на_мониторе=нет	Диагноз=проверить_видеокарту_кабель_и_монитор
Предполагаемая_причина=сбой_видеокарты_или_POST И Кулер_видеокарты_вращается=нет	Диагноз=проверить_видеокарту_и_ее_питание

Условие	Заключение
Состояние_загрузки=система_не_загрузилась И Сигналы_ошибки=нет	Диагноз=проверить_оперативную_память_и_загрузку
Предполагаемая_причина=проблема_с_блоком_питания И Розетка_работает=да	Диагноз=проверить_блок_питания_и_кабель
Предполагаемая_причина=проблема_с_блоком_питания И Розетка_работает=нет	Диагноз=проверить_розетку_и_сетевой_фильтр
Предполагаемая_причина=проблема_кнопки_или_системной_платы И Вентиляторы_вращаются=нет И Кнопка_включения_подключена=нет	Диагноз=проверить_подключение_кнопки_включения
Предполагаемая_причина=проблема_кнопки_или_системной_платы И Вентиляторы_вращаются=нет И Кнопка_включения_подключена=да	Диагноз=проверить_системную_плату
Компьютер_включается=да И Экран_черный=нет И Изображение_на_мониторе=да	Состояние_изображения=сигнал_изображения_есть
Состояние_изображения=сигнал_изображения_есть И Звук_загрузки=да	Диагноз=явной_неисправности_не_обнаружено
Компьютер_включается=да И Экран_мерцает=да	Симптом=изображение_мерцает
Симптом=изображение_мерцает И Изображение_на_мониторе=да	Предполагаемая_причина=нестабильная_работа_монитора_или_кабеля

Условие	Заключение
Предполагаемая_причина=нестабильная_работа_монитора_или_кабеля И Кабель_неплотно_подключен=да	Диагноз=проверить_подключение_кабеля
Предполагаемая_причина=нестабильная_работа_монитора_или_кабеля И Кабель_неплотно_подключен=нет	Диагноз=проверить_монитор

4. Пример работы и проверка

В базе знаний есть несколько цепочек для разных случаев: отсутствие изображения, сбой загрузки, проблемы с питанием и кнопкой включения, штатная загрузка и мерцание экрана. Ниже показан пример, в котором диагноз формируется последовательно через промежуточные факты.

Фрагмент цепочки для случая «компьютер включается, но изображения нет»:

Правило 1: ЕСЛИ Компьютер_включается=да И Экран_черный=да ТО Симптом=изображение_не_появляется

Правило 5: ЕСЛИ Симптом=изображение_не_появляется И Звук_загрузки=да ТО Предполагаемая_причина=система_загрузилась_без_вывода_изображения

Правило 8: ЕСЛИ Предполагаемая_причина=система_загрузилась_без_вывода_изображения И Изображение_на_мониторе=нет ТО Диагноз=проверить_видеокарту_кабель_и_монитор

Последовательность вопросов и вывода:

1) На вопросы «Компьютер_включается» и «Экран_черный» пользователь отвечает «да». Срабатывает правило 1, которое добавляет промежуточный факт «Симптом=изображение_не_появляется».

2) Пользователь отвечает «да» на вопрос «Звук_загрузки». Правило 5 использует факт из правила 1 и добавляет «Предполагаемая_причина=система_загрузилась_без_вывода_изображения».

3) На вопрос «Изображение_на_мониторе» пользователь отвечает «нет». Правило 8 использует заключение правила 5 и добавляет итоговый факт «Диагноз=проверить_видеокарту_кабель_и_монитор». В трассировке видна последовательность применённых правил 1, 5 и 8.

Другие цепочки: правила 2 → 3 → 11 ведут к проверке блока питания; правила 1 → 6 → 10 — к проверке загрузки и оперативной памяти; правила 17 → 18 → 19 или 20 помогают определить, следует ли проверить кабель или монитор при мерцании экрана. В каждой ветви промежуточное заключение используется следующим правилом.

4.1. Редактирование базы знаний

В главном меню можно просматривать правила, добавлять, изменять и удалять их. Изменения сохраняются в rules.txt. Например, добавление правила с новым условием «Корпус_перегревается» создаёт новый вопрос и позволяет получить новый диагноз без изменения программного кода.

4.2. Формирование вопросов по правилам

Программа ищет правила, ведущие к объекту «Диагноз», и раскрывает их условия через промежуточные заключения. Из неизвестных фактов выбирается вопрос из кратчайшей цепочки; при одинаковой длине приоритет определяется порядком правил в файле. Если добавить правило «ЕСЛИ Компьютер_включается=да И Корпус_перегревается=да ТО Диагноз=вероятный_перегрев_компьютера», система запросит новый признак и при нужных ответах сформирует новый диагноз.

5. Выводы

В ходе лабораторной работы разработана экспертная система на Python для диагностики компьютера. Правила хранятся в отдельном файле, а механизм прямого вывода добавляет промежуточные факты в рабочую память и передаёт их следующим правилам. Пример с правилами 1, 5 и 8 показывает, как несколько применений правил приводят к итоговому диагнозу.

Вопросы консультации строятся по текущей базе знаний, а не задаются фиксированным перечнем в программе. Редактирование rules.txt меняет возможные цепочки: новое условие может добавить вопрос, а новое правило с коротким путём к диагнозу может изменить результат консультации.

В базу включены несколько диагностических ветвей: по питанию, загрузке, выводу изображения, кнопке включения и мерцанию экрана. Трассировка показывает, какие правила сработали и какие промежуточные факты привели к выбранному диагнозу.

Приложение А

(Обязательное)

В приложении приведены основные фрагменты представления и разбора правил, прямого вывода, динамического выбора вопросов и сохранения изменений базы знаний.

1. Представление правила

```
@dataclass(frozen=True)
class Rule:
    conditions: tuple[tuple[str, str], ...]
    conclusion: tuple[str, str]


    def __str__(self) -> str:
        left = " И ".join(
            f"{name}={value}" for name, value in self.conditions
        )
        name, value = self.conclusion
        return f"ЕСЛИ {left} ТО {name}={value}"
```

2. Загрузка и разбор правил

```
def parse_rule(line: str) -> Rule:
    match = re.fullmatch(
        r"\s*ЕСЛИ\s+(.+?)\s+ТО\s+(.+?)\s*", line
    )
    if not match:
        raise ValueError("Ожидается формат: ЕСЛИ ... ТО ...")


    conditions = tuple(
        parse_pair(part)
        for part in re.split(r"\s+И\s+", match.group(1))
    )
    return Rule(conditions, parse_pair(match.group(2)))
```

3. Прямой вывод и обновление рабочей базы

```
for index, rule in enumerate(rules):
    if index in fired:
        continue
    if all(working_memory.get(name) == value
           for name, value in rule.conditions):
        fired.add(index)
        name, value = rule.conclusion
        if name not in working_memory:
            working_memory[name] = value
            kind = "итоговый диагноз" if name == GOAL else "промежуточный факт"
            trace.append(f"Сработало правило {index + 1}: добавлен {kind} {name}={value}")
            fired_this_pass = True

if GOAL in working_memory:
    break
if fired_this_pass:
    continue
```

4. Динамический выбор следующего вопроса

```
def next_question(rules, facts, goal=GOAL):
    paths = question_paths_for_goal(rules, facts, goal)
    candidates = [path for path in paths if path]
    if not candidates:
        return None
    return min(candidates, key=len)[0]
```

При добавлении правила функция add_rule сохраняет его в rules.txt. На следующей консультации функция next_question анализирует обновлённую базу. Например, после добавления условия «Корпус_перегревается» этот объект становится вопросом, а соответствующее правило может привести к новому диагнозу. Полный исходный код находится в файле main.py проекта.

Репозиторий: https://github.com/vovamadeinhell/expert-system-lab